

Git & GitHub Basics

Version control in a nutshell: track every change to your work, back it up in the cloud, and collaborate without emailing files named `final_v3_REALLY_FINAL.ipynb`.

1. What Are Git and GitHub?

Git is a program on your computer that records snapshots of a project over time, so you can see what changed, when, and by whom, and go back if needed. **GitHub** is a website that hosts Git repositories online: your backup, your portfolio, and the place where teams share code.

Repository	A project folder that Git is tracking ("repo" for short). The history lives in a hidden <code>.git</code> subfolder.
Commit	One saved snapshot of the project, with a message describing what changed. A repo's history is a chain of commits.
Stage	The waiting area for the next commit. You pick which changes go in with <code>git add</code> .
Remote	A copy of the repo hosted elsewhere, usually on GitHub. By convention it is named <code>origin</code> .
Clone	Your local copy of a remote repo, with its full history.
Push / Pull	Send your new commits up to GitHub / bring down commits that are on GitHub but not yet on your machine.

2. One-Time Setup

Tell Git who you are (this signs your commits) once per computer:

```
git config --global user.name "Ada Lovelace"
git config --global user.email "ada@example.com"
git --version          # confirm Git is installed
```

Authentication: the first time you push to GitHub, it will ask you to sign in. Follow the browser prompt (or use a personal access token as your password). This is also a one-time setup per computer.

3. Getting a Repository

`git clone https://github.com/user/repo.git` Download a repo from GitHub into a new folder, history included. This is how you get the course repo.

`git init` Start tracking the current folder as a brand-new repo (when you are creating a project from scratch).

4. The Everyday Cycle

Ninety percent of Git is this loop. Edit your files, then:



`git status` What changed? What is staged? Run this constantly; it also suggests the next command.

`git add report.ipynb` Stage one file for the next commit. `git add .` stages everything that changed.

`git commit -m "Add revenue analysis"` Save the snapshot with a short present-tense message describing the change.

git push	Upload your new commits to GitHub.
git pull	Download and apply commits from GitHub (do this before you start working, and to get instructor updates).

5. Looking at History and Changes

git log	The commit history: author, date, and message for each snapshot. Quit with q .
git log --oneline	The same, one commit per line. Much easier to scan.
git diff	Show exactly what you have changed since the last commit, line by line.
git diff --staged	Show what is staged, i.e. what the next commit will contain.

6. Undoing Things (The Safe Ones)

git restore report.ipynb	Throw away your uncommitted edits to a file and go back to the last committed version. (Cannot be undone; your edits are gone.)
git restore --staged report.ipynb	Un-stage a file you added by mistake. The edits themselves are kept.
git commit --amend -m "Better message"	Fix the message of the commit you just made (only if you have not pushed it yet).

7. Telling Git What to Ignore

A file named `.gitignore` in the repo's top folder lists patterns Git should never track: temporary files, secrets, giant datasets. One pattern per line:

```
.DS_Store      # macOS clutter
__pycache__/_  # Python cache folders
*.log          # any log file
data/raw/      # large data that belongs elsewhere
```

8. Putting a New Project on GitHub

```
# 1. On github.com: click "+" -> "New repository", name it, create it (empty).
# 2. In your project folder:
git init
git add .
git commit -m "First commit"
git remote add origin https://github.com/YOU/project.git
git push -u origin main      # -u links the branch; after this, plain "git push" works
```

Commit early, commit often. Small commits with clear messages ("Add tax column cleaning", not "stuff") make your history readable, and a pushed commit means your work survives a lost or broken laptop.

MSBA Bootcamp · Day 5 · Git & GitHub Basics — companion sheet to Command Line Basics; the daily loop to remember is `status -> add -> commit -> push`.